Kubernetes Ingress and Cloud Load Balancers

Day 15 | KUBERNETES / EKS / AKS / GKE

Use this guide to investigate a specific failure, apply a cause-specific change and verify recovery. Start with evidence from the affected workload. The examples require adaptation to your environment.

How the failure happens

Ingress is a routing API, while a controller turns that intent into a functioning data plane. Some controllers configure cloud L7 load balancers directly. Other designs expose a proxy inside the cluster through a cloud L4 load balancer. Understand which design your YAML targets.

Ingress Object Declares HTTP host and path routing. A matching controller must reconcile it.

Two Common Designs A controller can configure a cloud L7 load balancer, or a cloud LB can expose an in-cluster proxy.

This Reel Examples use cloud L7 integration. Service type LoadBalancer is a different API path.

Before you start

A healthy app and Service; exactly the intended cloud controller and class; scoped permissions; compatible pod networking; DNS and production TLS plan; cloud quota and cost awareness.

Diagnostic checks

```
apiVersion: v1
kind: Service
metadata: {name: api}
spec:
  selector: {app: api}
  ports:
  - {port: 80, targetPort: 8080}
  type: ClusterIP
```

Check The Service: api selects Pods labelled app=api. The application listens on 8080.

Check Readiness: No ready backends means a provisioned load balancer still cannot serve the app.

Apply the cause specific fix

Host Path Point To A Service

```
# Ingress spec rules fragment
rules:
- host: api.example.com
  http:
    paths:
    - path: /
      pathType: Prefix
      backend:
        service:
          name: api
          port: {number: 80}
```

Same Namespace: The Ingress and api Service must be in the intended namespace.

Dns Is Separate: Point api.example.com to the provisioned LB using the appropriate DNS record.

Aws Eks Alb Controller

```
# Ingress metadata + spec fragments
metadata:
  annotations:
    alb.ingress.kubernetes.io/scheme: internet-facing
    alb.ingress.kubernetes.io/target-type: ip
spec:
  ingressClassName: alb
  # Add the shared rules from the prior slide.
```

Prerequisites: Install/configure AWS Load Balancer Controller, IAM and subnet discovery. EKS Auto Mode uses its own supported setup.

Ip Target Example: The ALB registers routable Pod IPs. Validate CNI support, security rules and health checks.

Azure Aks Application Gateway

```
# Ingress metadata fragment for AGIC
metadata:
  annotations:
    kubernetes.io/ingress.class: azure/application-gateway
# Add the shared spec.rules.
# Requires AGIC + a configured Application Gateway.
```

Prerequisites: Enable or install AGIC, configure its identity and Application Gateway access, and provide backend connectivity.

Do Not Mix Controllers: These annotations are for AGIC. Application Gateway for Containers uses its own controller and configuration.

Gcp Gke Application Load Balancer

```
# Ingress metadata fragment for GKE
metadata:
  annotations:
    kubernetes.io/ingress.class: gce
# Add the shared spec.rules.
# Service annotation for a compatible NEG setup:
# cloud.google.com/neg: '{"ingress": true}'
```

Prerequisites: Use the GKE Ingress controller and a supported HTTP load balancing/network configuration.

Gke Difference: Use the documented gce class annotation. Container-native backends need compatible NEGs, ports and health checks.

Validate the result

```
kubectl describe ingress api
kubectl get service api
kubectl get endpointslices

curl -H "Host: api.example.com" \
  http://<load-balancer-address>/
```

Check The Result: Inspect events, controller logs and cloud LB backend health, then DNS, TLS and a real request.

Https: HTTP examples are for a lab. Configure the controller-specific certificate/listener path before production.

Practice exercise

Deploy the matching controller-specific example in a lab. Verify the Service endpoints before the load balancer, then test the host rule with an explicit Host header and finally real DNS/TLS.

Implementation notes

Use exactly the full manifest for your chosen controller; fragments are not standalone resources. Supply healthy app Pods, identity permissions, compatible networking, DNS and production TLS. The AWS example targets the standard AWS Load Balancer Controller with its alb class and routable IP targets. EKS Auto Mode requires its own supported configuration. The Azure example targets AGIC and classic Application Gateway, not Application Gateway for Containers. GKE uses its documented gce annotation and a compatible NEG-enabled Service; ingressClassName is not a replacement for that annotation. Cloud L7 backends may target Pod IPs directly rather than pass through ClusterIP. Provisioning a load balancer can incur charges. The examples do not provision controllers or deploy the application.

Interview question

Why can Ingress exist with no working URL? Missing controller ownership, cloud permissions, subnet configuration, unhealthy backends, DNS or TLS can each prevent success.

Record what changed

- The symptom and affected user operation.
- Evidence supporting the root cause and alternatives ruled out.
- The changed configuration and the version or identity involved.
- Positive and negative tests, plus any remaining risks.

Example status

Examples are educational templates or fragments. They were not deployed against a live cloud account or production cluster. A successful syntax check is not proof of runtime compatibility. Test adapted configurations in an isolated environment first.

Official references

Official documentation 1

Official documentation 2

Official documentation 3

Official documentation 4